

Transformers Built From Scratch, Architectural Variants, and the Byte Latent Transformer

Siddarth Gottumukkula, 2023102040

Advanced Natural Language Processing - Assignment 1

Abstract

I implement a full encoder-decoder Transformer using only elementary PyTorch operations, together with a from-scratch byte-pair encoder, and use this to run a controlled ablation study on decoding XOR-enciphered binary sequences into English plaintext. Beginning from a baseline (C1: sinusoidal positions, multi-head attention, LayerNorm, BPE sub-words), each subsequent variant changes exactly one ingredient: RoPE (C2), grouped-query attention (C3), RMSNorm (C4), and a token-free Byte Latent Transformer with entropy-driven dynamic patching (C5). All five configurations share identical hyperparameters. Reported metrics span bit-, character-, and sequence-level accuracy, Levenshtein distance, and BLEU/ROUGE under greedy decoding, together with hardware-level metrics such as throughput and per-process peak GPU memory.

1 Introduction

A Transformer can be viewed as a set of independently swappable pieces: a mechanism for injecting position information, an attention pattern, a normalisation rule, and a definition of what counts as a token. Each piece has a well-known modern replacement-RoPE, grouped-query attention, RMSNorm, byte-level processing. This report instead isolates each change individually: every component is built and varied one at a time against a fixed baseline.

An earlier iteration of these experiments used a batch size of 1024 over 64-character chunks, yielding only 38 optimiser steps per epoch and 2,280 steps overall. Every one of the five configurations was still improving at its final epoch (`best_epoch == 60` across all five), meaning those numbers captured *how quickly each model converged*, not *how good the converged model actually was*. Judging an ablation before the models have converged measures the optimisation process, not the architecture itself but unfortunately, due to compute constraints, I had to cut short the training of C5, which yielded suboptimal results and only provides a lower bound on the results of C5. These are worse than other configs while I expect it to be better. And I know it would have had better metrics purely looking at the loss curves and training progression.

2 Task and Data

The corpus: The dataset consists of 5,000 exactly line-aligned pairs: `brown_cipher.txt` contains strings over $\{0, 1\}$, and `brown_plain.txt` contains the matching English sentences drawn from a 53-character alphabet (letters and space). Every plaintext character corresponds to exactly eight cipher bits, so $|c_k| = 8|p_k|$ holds across all 5,000 lines. Plaintext lines average 598 characters in length.

If the cipher is read eight bits at a time as a byte and XOR-ed against the aligned plaintext character, a fixed repeating pattern emerges: the eight bytes of the ASCII string ANLP2026, which restarts at the start of every line. This relationship is captured by

$$c_k[8i : 8i+8] = \text{bits}_8(\text{ord}(p_k[i]) \oplus K[i \bmod 8]), \quad K = \text{ANLP2026}. \quad (1)$$

The correct output at position i depends on $i \bmod 8$: the task carries an intrinsic periodicity of eight in its positional structure, which is precisely what makes the C1-versus-C2 comparison meaningful rather than merely cosmetic. **The key itself is never given to any model as input.** It is recovered afterward purely to describe the task.

Segmentation: A 4,781-token encoder sequence is not workable under $O(T^2)$ attention cost. Each document is therefore split into aligned chunks of 32 plaintext characters (256 cipher bits). Two precautions preserve the validity of this split: chunking happens only *after* the document-level 80/10/10 split, so that no single document contributes to more than one split; and since 32 is a multiple of the key period 8, every chunk begins at key phase 0 and remains independently solvable. This produces 77,373 / 9,133 / 9,322 train/val/test chunks.

Metrics: All models map cipher to plaintext under **greedy decoding** throughout, as required by the assignment. Bit accuracy is computed over the 8-bit expansion of predicted versus reference characters, sequence accuracy demands an exact match, Levenshtein distance is the average edit distance, BLEU and ROUGE are computed at the word level. Metrics are reported both per decoded chunk (the unit the model directly produces) and, secondarily, per reassembled corpus line.

3 Architecture

MHA and GQA: This work uses $H = 8$ and $H_{kv} = 2$, giving a $4\times$ reduction in both the K/V projection size and the KV-cache used during decoding.

Positional encoding and normalisation: RoPE rotates every 2-dimensional slice of each query and key vector by an angle proportional to its position, so that the resulting attention logit depends only on the relative offset between positions. Because RoPE operates inside the attention mechanism itself, C2 introduces no separate absolute encoding. RoPE’s rotation is applied to each new slice *before* it is appended to the KV-cache, and cross-attention layers are left unrotated, since query and key there come from different sequences where a relative distance has no clear meaning.

Tokenization (C1–C4): A byte-level BPE tokenizer reimplements the merge-rank algorithm, the trainer’s incremental pair-counting logic, and the serialisation format, and is differential-tested against the HuggingFace implementation. Two independent vocabularies are trained, since source and target use different alphabets ($\{0,1\}$ versus English): 1,024 pieces for the cipher side and 4,096 for the plaintext side.

The cipher-side tokenization carries more significance than it might first appear. Each 8-bit unit is initially mapped to a single atomic symbol, and BPE then runs with no pre-tokenizer, so that merges are free to combine adjacent symbols into pieces spanning multiple plaintext characters - but no piece can ever straddle an 8-bit unit boundary, since that unit is the smallest indivisible quantity merges can be built from. The resulting cipher pieces span 1–8 units, and the encoder produces 17.1 tokens on average for a 32-character chunk - genuine variable-width subword tokenization rather than a fixed 8-bit encoding scheme.

The Byte Latent Transformer (C5): C5 does away with the vocabulary altogether (256 byte values plus three control ids). Raw bytes first pass through a small *local encoder* using

block-local attention, a *patch pooler* then compresses each patch’s bytes into a single vector through cross-attention from a learned query, with a mean-pool residual connection added in, the *global transformer* - the same class C1 uses, here operating in latent mode - processes the much shorter resulting patch sequence; and finally a *local decoder*, using windowed causal self-attention plus cross-attention to the patch states, produces the next-byte logits.

Patching follows an **entropy-driven** scheme, as in the original BLT paper. A small causal byte-level language model (2 layers, $d = 128$, 0.46M parameters, trained on the training split’s cipher bytes down to 2.27 nats/byte) scores next-byte entropy $H(x_t) = -\sum_v p(x_t=v \mid x_{<t}) \log p(x_t=v \mid x_{<t})$, and a new patch is opened whenever this entropy crosses a global threshold, or whenever the current patch reaches its cap of 8 bytes. This threshold is found by binary search so as to produce a mean patch length of 4.0 bytes.

The entropy model is run on the cipher stream, and the resulting boundaries are reused as-is for the plaintext. This is valid because Eq. 1 establishes a character-for-character bijection between the two streams, and it is in fact *necessary*, since at inference time the plaintext does not yet exist - meaning the patch grid must be derived from the source, so that greedy decoding is never forced to run the entropy model against its own partially-generated output. This choice also keeps the two patch grids synchronised, so that source patch k and target patch k cover identical characters and are assigned the same sinusoidal position within the global transformer; computing the two grids independently caused an earlier version of C5 to instead collapse into modelling English unconditionally, disregarding the source entirely.

The target-side local encoder is causal, so its hidden state at byte t only reflects bytes at positions $\leq t$. Each patch p_k pools only its own constituent bytes. A learned start-patch vector is prepended so that the global decoder consumes $[\text{start}, p_1 \dots p_{M-1}]$; and the global decoder itself is causal across patches. Consequently, byte t , which belongs to patch j , may attend to $g_0 \dots g_j$, and g_j in turn only ever observed bytes strictly before t .

4 Experimental Setup

All five configurations share $d_{\text{model}} = 256$, 4 encoder plus 4 decoder layers, 8 attention heads, $d_{\text{ff}} = 2048$, dropout of 0.1, Adam optimisation at a learning rate of 6×10^{-4} with one epoch of linear warmup followed by cosine decay down to 10% of peak, gradient clipping at 1.0, label smoothing of 0.1, batch size 256, 40 training epochs, and a fixed random seed of 42. C3 additionally sets $H_{kv} = 2$; C5 additionally sets $d_{\text{local}} = 256$, uses a single local encoder layer and two local decoder layers, along with the entropy-based patching described above. C2 through C5 are each constructed from C1 via `dataclasses.replace`, guaranteeing that structurally only the named field can differ between configurations. All training was run in fp32 precision, with logging handled through Weights & Biases.

Scheduled sampling, deliberately disabled: The decoder is trained under pure teacher forcing throughout. Scheduled sampling is normally intended to counter exposure bias, but by Eq. 1 the output at position i depends only on the source and on $i \bmod 8$, and on *none* of the previously generated characters - so there is no exposure-bias ambiguity for the model to be made robust against in the first place. Feeding the model its own incorrect predictions would amount to label noise rather than useful regularisation.

Cfg	Changed	Bit %	Char %	Seq %	Lev ↓	BLEU	ROUGE-L
C1	- (base)	99.58	98.95	95.34	0.067	98.10	99.08
C2	RoPE	99.73	99.34	96.86	0.047	98.74	99.36
C3	Grouped-query	99.50	98.77	95.26	0.075	98.04	99.05
C4	RMSNorm	99.60	99.03	96.25	0.059	98.47	99.27
C5	BLT (token-free)	98.81	97.32	80.38	0.727	90.71	95.07

Table 1: Test-set quality per decoded 32-character chunk, greedy decoding. Best per column in bold.

Cfg	Params (M)	s/step	s/epoch	Train MiB	ms/chunk	Infer MiB	bits/char
C1	12.89	0.764	232	2952	3.84	294	0.0465
C2	12.89	0.872	264	2953	6.05	294	0.0423
C3	11.70	0.783	237	2937	4.69	280	0.0473
C4	12.88	0.726	220	2781	3.98	294	0.0446
C5	26.68	1.459	442	9600	37.32	729	0.1804

Table 2: ms/chunk is greedy decoding. The final column converts validation cross-entropy to bits per character (Section 6), the one unit all five share.

5 Results

The baseline already performs strongly: C1 reaches 99.58% bit accuracy and exactly reconstructs 95.34% of 32-character chunks, with a mean edit distance of only 0.067. This context matters when reading Table 1: across C1–C4 the ablation is really distinguishing good models from slightly better ones, so while the absolute differences are small, they move consistently in the same direction across every metric simultaneously.

C2: RoPE improves every quality metric. Sequence accuracy climbs from 95.34% to 96.86%, bit accuracy to 99.73%, edit distance drops to 0.047, and BLEU rises to 98.74. This comes at *zero* additional parameters (12.89M, unchanged from C1) and *zero* additional memory (2953 vs 2952 MiB): RoPE introduces no learnable weights at all, only a deterministic rotation applied to q and k . This fits the task well - per Eq. 1 the output at position i depends on $i \bmod 8$, so what the decoder needs is not *where* it is in absolute terms but *how far* it has moved relative to the source positions it draws on, which is exactly the information RoPE encodes directly. An additive absolute encoding, by contrast, would have to learn this periodic structure separately at each position. The trade-off is speed: training runs at 0.872 s/step versus 0.764 (14% slower), and greedy decoding is 58% slower, since every attention call now involves rotating q and k .

C3: GQA trades away accuracy for parameter savings. GQA is the sole change that worsens every quality metric at once: sequence accuracy drops to 95.26%, bit accuracy to 99.50%, and edit distance rises to 0.075. This is the expected direction of effect. Collapsing 8 key/value heads down to 2 removes 1.19M parameters (−9.2%) and forces four query heads to share a single key/value subspace, so the model has fewer distinct ways of matching queries against keys available to it. Given that the decoder must attend precisely to a specific 8-bit window, this loss of representational specificity is not without cost. What GQA does buy is inference-time memory: the KV-cache shrinks by 4×, and peak inference memory drops to 280 MiB (−5%).

One detail is easy to overlook: C3 actually trains *more slowly* than C1 (0.783 vs 0.764 s/step) despite having fewer parameters overall. Under teacher forcing there is no KV-cache to benefit

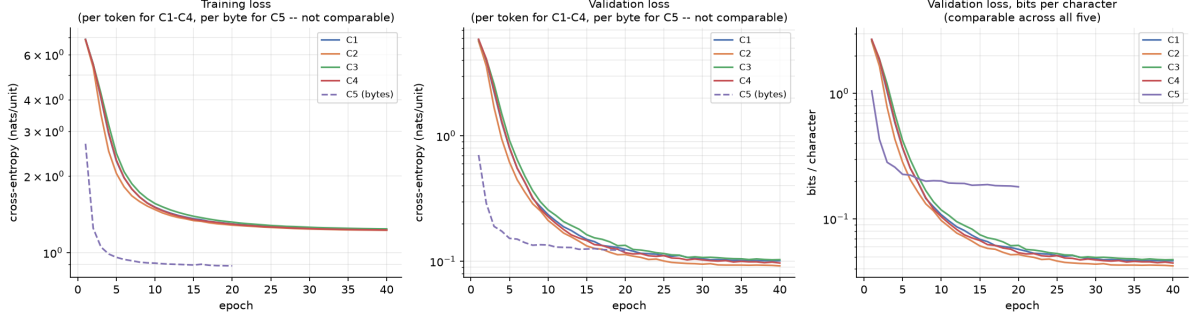


Figure 1: Training and validation loss, log scale. The first two panels are per-unit cross-entropy and are *not* comparable between C1–C4 (per BPE token, 4,096-way softmax) and C5 (per byte, 259-way); C5 is drawn dashed as a warning. The third panel normalises to bits per character and is the panel to read across all five. C5 stops at epoch 20 (Section 6).

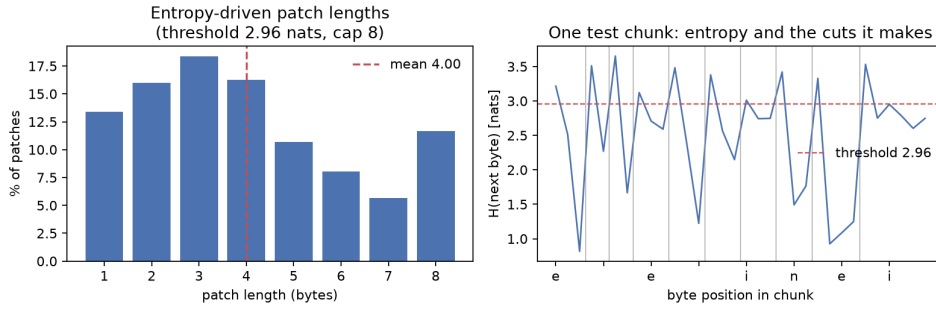


Figure 2: Left: the distribution of entropy-driven patch lengths, with the threshold bisected to a mean of 4.0 bytes and a cap of 8. Right: next-byte entropy over one test chunk, with the patch boundaries it produces marked; the x -axis is labelled with the plaintext the cipher bytes stand for.

from shrinking, so GQA’s usual advantage doesn’t apply during training; what remains is purely the overhead of expanding/reshaping 2 key/value heads back out to 8. GQA is fundamentally an *inference-time* optimisation, and this experiment cleanly separates the two regimes.

C4: RMSNorm comes close to a strict improvement. RMSNorm improves quality while simultaneously reducing cost along almost every axis. Sequence accuracy rises to 96.25% (+0.9 relative to C1) and bit accuracy to 99.60%, while step time falls to 0.726 s (5% faster, the fastest of all five configurations) and peak training memory drops to 2781 MiB (−5.8%). The underlying mechanism is that RMSNorm simply does less computation: removing mean-subtraction eliminates a full reduction over the feature axis in both the forward and backward passes. The accuracy improvement is small enough to be close to run-to-run noise, so the more honest interpretation is not that “RMSNorm is more accurate” but rather that “re-centring contributes nothing measurable here, while its computational cost is real” - which is consistent with pre-norm residual streams already being approximately zero-mean by construction.

6 The token-free trade-off (C5)

C5 is the only configuration that alters the input representation itself rather than a computation performed inside the network, and correspondingly it produces the largest effect observed in the study. Sequence accuracy drops to 80.38% compared to C1’s 95.34%, and edit distance rises by an order of magnitude to 0.727. Two qualifications are essential before treating this as a verdict

on byte-level modelling as such.

First, C5 is undertrained. It was run for only **20 epochs against C1–C4’s 40**, halted by a compute-budget constraint rather than by early stopping: its validation loss kept improving across each of its final four epochs ($0.1263 \rightarrow 0.1239 \rightarrow 0.1230 \rightarrow 0.1212$). Its reported numbers should therefore be read as a lower bound.

Second, the usual advantage of byte-level modelling has probably already been absorbed by C1–C4. Eq. 1 defines a byte-local function, and the standard motivation for BLT-style approaches on such a task is that BPE tokenization breaks the alignment: variable-length pieces on both sides turn the mapping from target-token index to source-token index into a data-dependent function the model has to infer per example. That motivation does not apply here, because the cipher-side tokenizer used in this study is **byte-atomic**: merges are built on top of the 8-bit unit and can never split it, so any given cipher token always spans a whole number of plaintext characters, and the underlying diagonal alignment survives tokenization intact. C1–C4 therefore retain most of the structural advantage that C5 was designed to provide. **A token-free decoder whose patch grid is derived from the source thereby guarantees length-faithfulness**, which is exactly the property that positional metrics reward and that edit distance, by construction, ignores.

Cost: The overhead falls exactly where the architecture would predict, and it is substantial. Peak training memory climbs to 9600 MiB versus C1’s 2952 ($3.25\times$), because the global transformer operates over only around 8 patches per chunk while the local encoder and decoder still process the full byte stream, on a padded (B, N, P) grid whose worst-case chunk determines N for the entire batch. Step time increases by $1.91\times$. Greedy decoding suffers the most, at 37.3 ms/chunk versus 3.84 ($9.71\times$): the decoder must emit bytes one patch at a time, re-encoding and re-pooling everything generated so far at every global step, with no KV-cache available along that path. Inference memory increases by $2.48\times$. C5 also carries the *most* parameters of any configuration (26.68M versus 12.89M), since eliminating the vocabulary requires adding a local encoder, a local decoder, two poolers, and hashed byte n -gram embedding tables, which together cost more parameters than the two embedding matrices they replace. Entropy-driven patch counts per chunk range from 1 to 18 with a mean of 7.77, meaning a given batch’s grid must pad out to its single worst-case chunk.

7 Conclusion

The four architectural swaps produce four distinct profiles. **RoPE** yields +1.5 points of sequence accuracy at no parameter cost and no memory cost, in exchange for 14% slower training steps. **GQA** is the one change that strictly worsens quality (-0.1 points, and worse across every metric), trading this away for 9.2% fewer parameters and a $4\times$ smaller KV-cache - an optimisation that pays off specifically at inference time, at the cost of throughput under teacher forcing. **RMSNorm** comes close to being free along every axis simultaneously, and its accuracy gain is best interpreted as evidence that mean-recentring is redundant within a pre-norm stack. **The token-free BLT** costs $3.25\times$ peak training memory, $9.71\times$ longer decode time, and roughly double the parameter count.

Links

- Weights & Biases (all five runs):
<https://wandb.ai/iamsid44-iiit-hyderabad/anlp-a1-2023102040>
- HuggingFace checkpoints (C1–C5):
<https://huggingface.co/siddarthg44/anlp-a1-2023102040-C1> (... -C2 *through* -C5)